



User Manual

WORKS WITH



Home Assistant



MQTT



Domoticz



openHAB



www.lemcke.nl

Contents

1. Preface	3
2. Installation	3
3. Configuration	3
4. Configure an already-connected module	7
5. Buttons	8
6. Home Assistant integration	9
7. MQTT	12
8. Domoticz configuration	13
9. OpenHAB configuration	15
10. Firmware updates	16
11. Upgrading from an older firmware version	16
12. Programming the module with custom firmware	17
13. Using Salt Sentry with ESPHome	19

1. Preface

This is the manual for the Salt Sentry. The Salt Sentry measures how much salt is still present in a water softener and reports this to a home automation system. Currently supported: Home Assistant, Domoticz, and OpenHAB. Other systems that do support MQTT can also be used.

This makes it possible, for example, to get a notification on your phone when the salt is almost empty, so you no longer have to check manually and never forget to refill again.

2. Installation

Mount the Salt Sentry above the salt block in the water softener using the included piece of double-sided tape. The "eye" of the Salt Sentry (the sensor) should point downward, toward the salt.

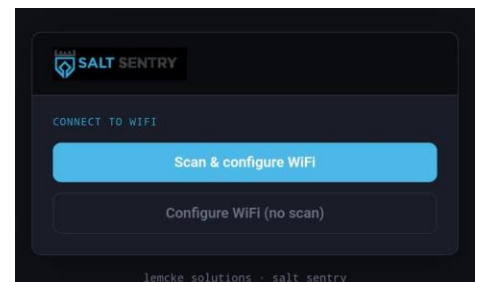
The Salt Sentry is powered via a USB adapter and a micro USB cable. Depending on the type of softener, you may need to make a small cut-out in the housing to route the cable through.

Note: The sensor gives the most reliable readings with salt blocks or pellets. Loose salt crystals can produce inaccurate readings, because the crystals can give inaccurate reflections.

3. Configuration

As soon as the Salt Sentry is powered on for the first time, it starts a WiFi network called "saltSentry". Connect to this network with, for example, your phone. Most modern phones will automatically open the configuration page; if not, browse to <http://192.168.4.1>.

Choose "Scan & configure WiFi" to search for and connect to available networks, or "Configure WiFi (no scan)" if you prefer to type the network name manually.



The full configuration screen. All sections are collapsed by default except WiFi Credentials.

Select your WiFi network and enter the password. Below that, several collapsible sections let you configure the Salt Sentry for the systems you use. You only need to fill in the sections that apply to you, everything else can be left empty.

Note: Using Home Assistant? You only need to fill in the WiFi credentials during setup, the Home Assistant integration handles everything else automatically. The sections below (MQTT, Domoticz, OpenHAB) are only needed if you want to use one of those systems instead of, or in addition to, Home Assistant.

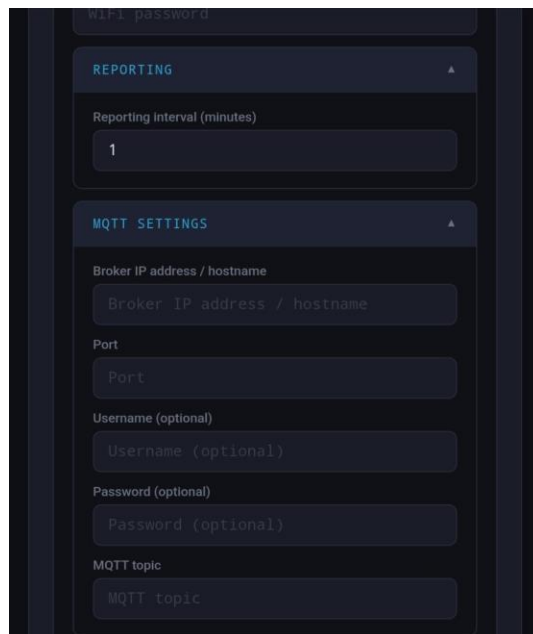
Reporting

In the reporting section, you can configure how often the Salt Sentry takes a measurement and reports it to all configured systems. The default is 1 minute.

MQTT Settings

If you want the Salt Sentry to send an MQTT message (for example to a home automation system that supports MQTT like Homebridge), fill in the broker IP address / hostname, port, and MQTT topic. Username and password are optional and only needed if your broker requires authentication.

The salt percentage is published to the configured topic. The distance in cm (or inch, depending on your unit setting) is published on the same topic with the postfix "_distance".



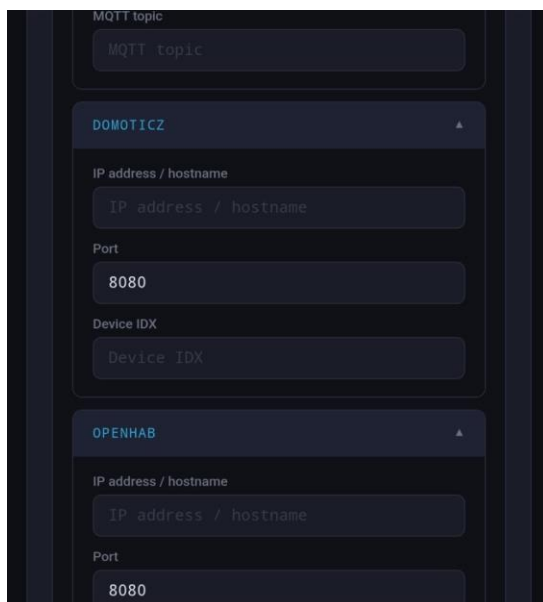
The screenshot shows the 'MQTT SETTINGS' section of a configuration interface. It includes a 'REPORTING' section above it with a 'Reporting interval (minutes)' set to 1. The 'MQTT SETTINGS' section contains the following fields: 'Broker IP address / hostname' (placeholder: Broker IP address / hostname), 'Port' (placeholder: Port), 'Username (optional)' (placeholder: Username (optional)), 'Password (optional)' (placeholder: Password (optional)), and 'MQTT topic' (placeholder: MQTT topic).

Domoticz

If you want to use the Salt Sentry with Domoticz, fill in the Domoticz IP address / hostname, port (default 8080), and the Device IDX of the virtual sensor you created in Domoticz. See chapter 8 for how to set up the Domoticz device.

OpenHAB

If you want to use the Salt Sentry with OpenHAB, fill in the OpenHAB IP address / hostname, port (default 8080), and the Item ID of the item you created in OpenHAB. See chapter 9 for how to set up the OpenHAB items.



The screenshot shows the 'DOMOTICZ' and 'OPENHAB' sections of a configuration interface. The 'DOMOTICZ' section includes fields for 'IP address / hostname' (placeholder: IP address / hostname), 'Port' (set to 8080), and 'Device IDX' (placeholder: Device IDX). The 'OPENHAB' section includes fields for 'IP address / hostname' (placeholder: IP address / hostname) and 'Port' (set to 8080). Above the Domoticz section is an 'MQTT topic' field with a placeholder 'MQTT topic'.

Water Softener Settings

Warning: These settings have no effect when using the Home Assistant integration. They are only needed for MQTT, Domoticz, or OpenHAB, Home Assistant users configure the water softener model and distances directly in Home Assistant (see chapter 6).

Measure from the sensor down to the salt surface. Enter the distance when full as Full, and when empty as Empty. Full must always be smaller than Empty.

Choose your unit: cm or inch. The current live sensor reading is shown for reference, with a refresh button.

Water softener model.

Choose your model from the dropdown to automatically fill in the Full and Empty distances. You can still adjust the distances afterwards, even when a preset model is selected. Choose "Other (manual entry)" if your model isn't listed and enter the distances yourself.

This list is still rather small. If you find good values for your own water softener that is not yet in the list, feel free to send us a message so that we can add it for other users.

Use the calibration offset to fine-tune readings if the sensor is consistently off. A positive value increases the reported distance, a negative value decreases it. In most cases this is not needed. Leave it at 0 if unsure.

Steady Read

When enabled, the Salt Sentry takes three measurements per reporting cycle and uses the median value to remove single-sample outliers. It also holds back small upward jumps in salt level (up to 2 cm), these are treated as sensor jitter rather than real change. Normal salt usage and larger drops (a refill) are always reported immediately. This applies to every place the salt level is reported, including the live status used by integrations like Home Assistant, keeping automation graphs smooth without masking real events. Recommended for most installations.

After pressing "Save & connect", the Salt Sentry will try to connect to the configured WiFi network. If this fails, it returns to access point mode so you can try again.

Once connected, the Salt Sentry immediately sends a first measurement, and will keep reporting on the configured interval.

The screenshot shows a mobile application interface for configuring a water softener. At the top, there are dropdown menus for 'DOMOTICZ' and 'OPENHAB'. Below these is a section titled 'WATER SOFTENER SETTINGS'. A red warning box states: 'Note: These settings have no effect when using the Home Assistant integration. They are only needed for MQTT, Domoticz or OpenHAB.' Below the warning, a text box explains: 'Measure from the sensor down to the salt surface. Enter the distance when full as Full and when empty as Empty. Full must always be smaller than Empty.' There are two radio buttons for 'Unit': 'cm' (selected) and 'inch'. A 'Current sensor reading' is displayed as '128.20 cm' with a refresh icon. Below this is a 'Water softener model' dropdown menu set to 'Other (manual entry)'. There are two input fields for 'Full - distance (cm)' and 'Empty - distance (cm)'. A text box explains the 'calibration offset' and provides a '0' input field. At the bottom is a large blue button labeled 'Save & connect'.

4. Configure an already-connected module

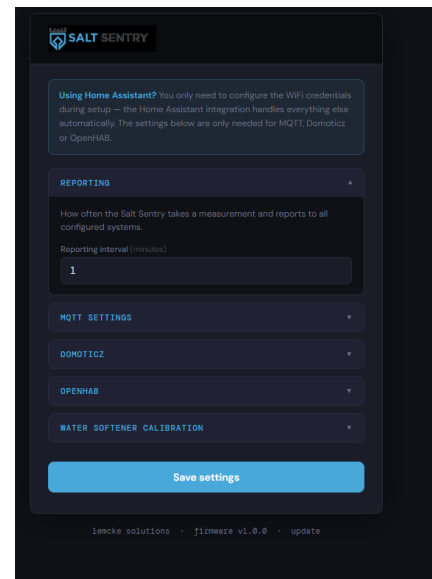
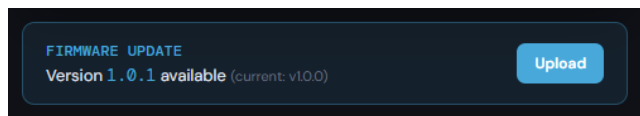
If you want to change the configuration or check the status of the Salt Sentry after it is already connected to your WiFi network, you can do this by browsing to the IP address of the Salt Sentry.

If you don't know the IP address, press the right button on the Salt Sentry (see chapter 5). This opens a temporary access point called "Salt sentry online". Connect to it with your phone to retrieve the current IP address, then reconnect to your own WiFi network and browse to that address, for example <http://192.168.2.46>.

This page looks almost identical to the initial captive portal (chapter 3), and offers the same settings, sections, and Water Softener Calibration options.

At the bottom of the page you can see the installed firmware version, and a link to update it.

If new firmware is available, a notification is displayed here as well; see chapter 10 for more information about firmware updates.



5. Buttons

The Salt Sentry has 2 buttons that can be pressed with a narrow object, for example a small screwdriver or paper clip.

- The left button resets the module without clearing the settings.
- The right button opens a temporary access point named "Salt sentry online" for 5 minutes, so you can find the IP address of the module without re-entering setup mode (see chapter 4).
- Holding the right button for 2.5 seconds performs a factory reset: all settings are erased and the module returns to setup mode.



Warning: A factory reset cannot be undone. All WiFi, MQTT, Domoticz, OpenHAB, and water softener settings will be permanently erased.

6. Home Assistant integration

The Salt Sentry has a dedicated Home Assistant integration that automatically discovers the device on your network and exposes it as sensors, without needing MQTT or any manual YAML configuration.

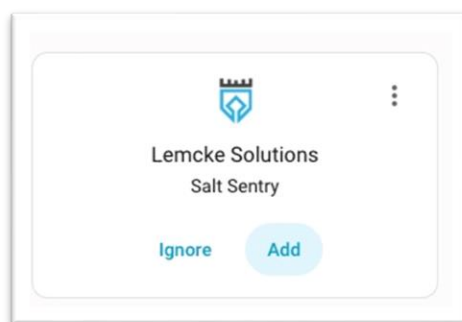
Installation

The Salt Sentry integration is available in the HACS.

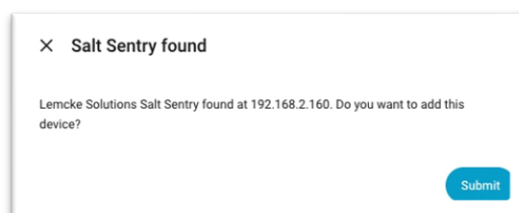
- Open HACS in Home Assistant.
- Search for "Salt Sentry".
- Install the integration and restart Home Assistant when prompted.

Discovery and setup

Once installed, Home Assistant automatically discovers any Salt Sentry on the same network via Zeroconf (mDNS), no manual steps are required to find it. A discovery card appears under Settings → Devices & services.



You can also add the device manually via Settings → Devices & services → Salt Sentry → add, and enter the IP address yourself.

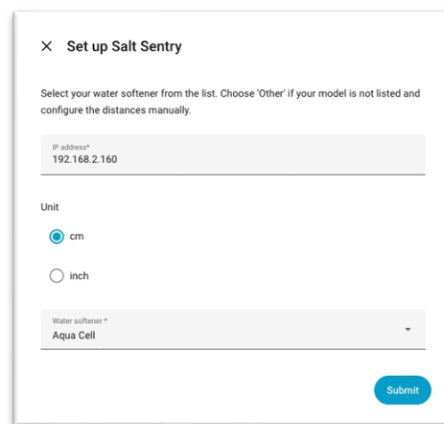


During setup you configure:

- IP address of the device (prefilled automatically when discovered)
- Unit: cm or inch
- Water softener model: choose from the list to prefill the distances, or "Other" to enter them manually

On the next screen you confirm or adjust:

- Distance when full and distance when empty, prefilled based on the chosen model, but always adjustable



- Scan interval, how often Home Assistant polls the device, in minutes (default: 1)

×

Set distances

Settings for Aqua Cell. A small distance means a full tank (a lot of salt), a large distance means an empty tank.

Current measurement: 25.3 cm

Distance when tank is full*

5

Distance when tank is empty*

35

Update interval (minutes)*

1

Submit

Note: The calibration offset is not part of the initial setup. It can be set afterwards via Settings → Devices & services → Salt Sentry → Configure, together with the unit, model, and scan interval.

Device and entities

In meterkast

SALT SENTRY

Device info

Salt Sentry

by Lemcke Solutions

Firmware: 1.0.0

Salt Sentry

Visit

Related

Automations

Scripts

No scripts

Sensors

↔ Distance 25.3 cm

❖ Salt level 32.3%

Add to dashboard

Configuration

Firmware Up-to-date

Add to dashboard

Diagnostic

Hardware revision A

+1 disabled entity

Add to dashboard

Activity

Today - September 2, 2026

Firmware Unavailable 10:59:00 AM

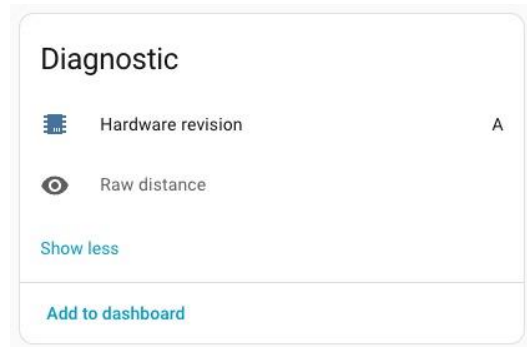
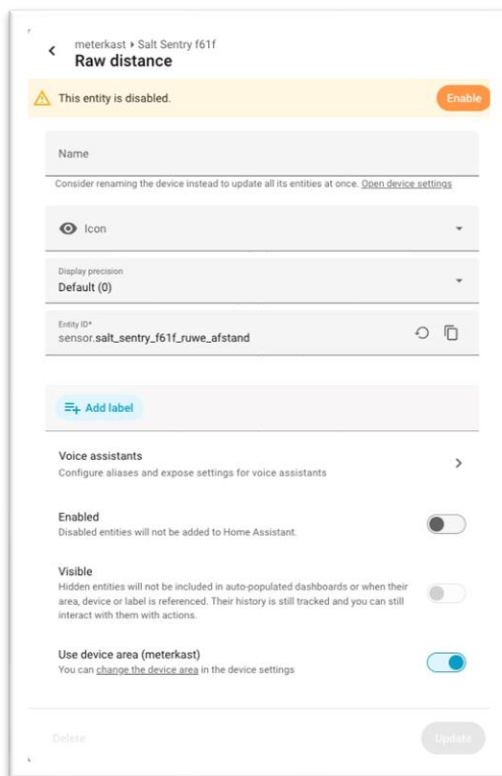
Hardware revision Unavailable 10:59:00 AM

Entity	Category	Description
Salt Level	N/A	Salt percentage (0-100%)
Salt Distance	N/A	Corrected distance measurement
Salt Distance raw	Diagnostic	Raw distance from the sensor, without correction (disabled by default)

Entity	Category	Description
Hardware Revision	Diagnostic	Hardware revision of the device
Firmware	N/A	Update entity for OTA firmware updates (see chapter 10)

The "Salt Distance raw" entity is disabled by default. It is only useful if you have set a calibration offset and want to see what the sensor measures before correction. In the Diagnostic section of the device page, click "Show more" to reveal it.

Then open the "Raw distance" entity and click Enable.



Enabling the "Raw distance" diagnostic entity.

Reconfiguring or removing

To change the unit, water softener model, distances, scan interval, or calibration offset after setup, go to Settings → Devices & services → Salt Sentry → Configure.

If the IP address of the device changes, you can update it without removing the integration via the same screen (Reconfigure).

To remove the integration entirely: Settings → Devices & services → find Salt Sentry → three-dot menu → Delete.

Automation examples

Notify when the salt level is low:

```
automation:
- alias: "Salt level low notification"
  trigger:
    - platform: numeric_state
      entity_id: sensor.salt_sentry_salt_level
      below: 20
  action:
    - action: notify.notify
      data:
        title: "Salt Sentry"
        message: "Salt level is below 20%. Time to refill!"
```

Weekly salt level report:

```
automation:
- alias: "Weekly salt level report"
  trigger:
    - platform: time
      at: "09:00:00"
      weekday: mon
  action:
    - action: notify.notify
      data:
        title: "Salt Sentry"
        message: "Current salt level: {{ states('sensor.salt_sentry_salt_level') }}%"
```

Troubleshooting

Device not found during setup confirm the IP address is correct and the device is powered on, and check that Home Assistant and the Salt Sentry are on the same network or VLAN.

Salt level seems incorrect go to Configure and re-enter the full and empty distances after measuring with the device in place. Check the "Salt Distance raw" diagnostic entity to see what the sensor actually reports, and use the calibration offset to fine-tune.

Entity shows unavailable the device could not be reached during the last poll. Check power and network connectivity, Home Assistant recovers automatically once the device is reachable again.

Distance and salt level show unknown while reachable the device is reporting an out-of-range or invalid measurement (for example the tank lid is open, or there is nothing for the sensor to measure against). Values recover automatically once a valid reading comes in.

7. MQTT

Note: If you use Home Assistant, you do not need to use MQTT, use the native Home Assistant integration described in chapter 6 instead. MQTT is useful if you want to connect the Salt Sentry to a different system, or your own MQTT-based setup.

Fill in the MQTT Settings section during configuration (chapter 3) with your broker's IP address / hostname and port. Username and password are optional.

The salt percentage is published (retained) to the configured MQTT topic. The distance is published (retained) on the same topic with the postfix "_distance", in the unit configured on the device (cm or inch).

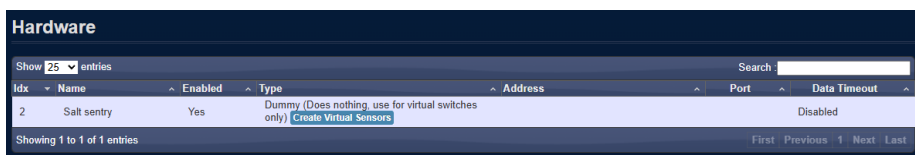
Note: The Salt Sentry also publishes an online/offline status message on the topic name with the postfix "_status", using MQTT's Last Will and Testament, useful if you want to monitor whether the device is currently connected.

8. Domoticz configuration

Note: Authenticate the Salt Sentry against Domoticz using Trusted Networks rather than a username and password login.

To use the Salt Sentry with Domoticz, a device must first be created, and you need to create two virtual sensors on it: one for the salt percentage, and one for the measured distance.

Go to Setup → Hardware in Domoticz and create a Dummy hardware device. Then click "Create Virtual Sensors" on that device.

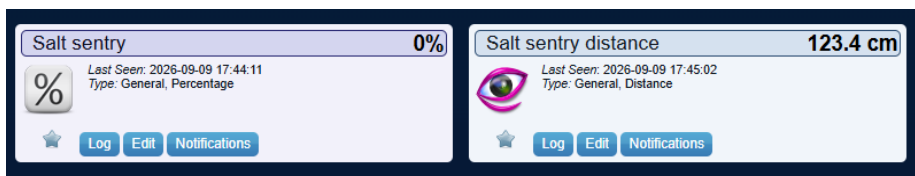
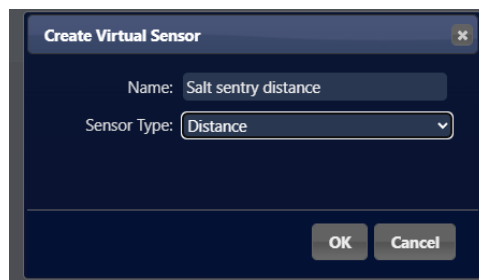
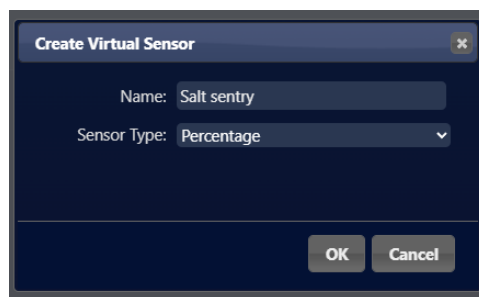


The Dummy hardware device, with the "Create Virtual Sensors" button.

First create a sensor of type Percentage, for the salt level:

To connect the Salt Sentry to this device, find its IDX under Setup → Devices, and enter it in the Domoticz section of the Salt Sentry's configuration page (chapter 3).

Every time the Salt Sentry takes a measurement, it also sends the measured distance. To receive this in Domoticz, create a second virtual sensor of type "Distance" on the same device:



Both sensors appear on the "Utility" tab: the percentage on the left, the distance on the right.

Note: The distance is sent in cm or inch, depending on the unit configured on the device (chapter 3). Only the number is sent, not the unit itself, so make sure the Domoticz sensor's expected unit matches what the Salt Sentry is configured to send.

Note: The Salt Sentry assumes the distance sensor's IDX is the percentage sensor's $\text{IDX} + 1$. For example, if the percentage sensor has IDX 23, the distance is sent to IDX 24. Make sure to create the distance sensor directly after the percentage sensor so the IDX numbers line up.

9. OpenHAB configuration

To use the Salt Sentry with OpenHAB, create 2 items via Settings → Items in the OpenHAB UI, no YAML file is needed.

- An item with the Item ID you will enter in the Salt Sentry's configuration (chapter 3), for the salt percentage.
- A second item with that same name followed by "_distance", for the distance measurement.

Both items should be created with type Number.

The Salt Sentry sends the measured distance in the unit configured on the device (cm or inch).

Create New Item •

Name	SaltSentry	ⓘ
Label	Item label for display purposes	
Type	Number	>
Dimension		>
Icon	temperature, firstfloor...	
Semantic Value	None	>
Non-Semantic Tags	0	
Add tag		
Parent Groups	0	
Select		

Create Cancel

Note: Item states are not updated in real-time. Click the refresh button to update.

2 Items		
Filter		
S	SaltSentry Number	40 >
S	SaltSentry_distance Number	10.70 >

The two items in the OpenHAB UI: "SaltSentry" holds the salt percentage, "SaltSentry_distance" holds the measured distance (in cm or inch, depending on the configured unit).

Once the items exist, fill in the OpenHAB IP address / hostname, port, and Item ID in the OpenHAB section of the Salt Sentry's configuration page.

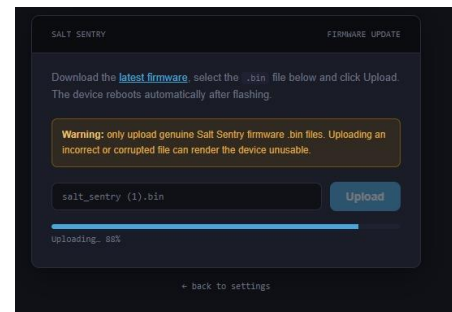
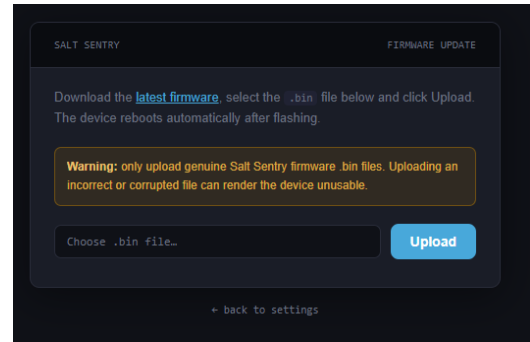
10. Firmware updates

There are two ways to update the Salt Sentry's firmware without reprogramming it from scratch:

Via the device web interface

Browse to the Salt Sentry's configuration page and open the "Firmware update" page (linked from the settings page footer, chapter 4).

Download the latest firmware release .bin file from GitHub (github.com/Lemcke-solutions/salt-sentry-firmware-hwa/releases), select it, and click Upload. The device reboots automatically after flashing, this can take up to about 2 minutes.



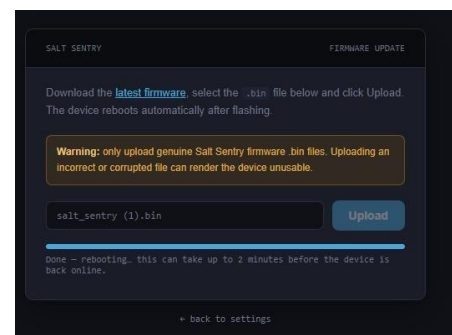
Warning: Only upload genuine Salt Sentry firmware .bin files. Uploading an incorrect or corrupted file can render the device unusable.

While uploading, a progress bar shows the upload percentage.

Once the upload completes, the device flashes the new firmware and reboots, this can take up to about 2 minutes before it is back online.

Via Home Assistant

If you use the Home Assistant integration (chapter 6), firmware updates can be installed directly from Home Assistant. The Firmware entity checks GitHub for new releases automatically every 6 hours; when an update is available, you can install it with one click from the device page or the Updates dashboard, without needing to visit the device's web interface.



Note: Firmware updates, whether installed through the device or through Home Assistant, require the device to be reachable on the local network at the time of installation.

11. Upgrading from an older firmware version

If you have an older Salt Sentry (ordered before august 2026) that does not have the web-based firmware update page described in chapter 10, or if you've flashed it with ESPHome and want to go back to the stock firmware, you can flash the new firmware over USB instead, using a tool called esptool.

USB drivers

The Salt Sentry uses a CH340 USB-to-serial chip. If your computer does not recognize the module when you connect it, install the CH340 driver for your operating system first.

- Windows and macOS: download the driver from the WCH website (wch-ic.com/downloads/CH341SER_ZIP.html) and install it, then reconnect the module.
- Linux: the CH340 driver is normally built into the kernel already; no action should be needed.

Installing esptool

esptool is a command-line tool for flashing Espressif chips such as the ESP8266 used in the Salt Sentry. It requires Python (3.7 or newer). Install it with:

```
pip install esptool
```

Finding the serial port

Connect the Salt Sentry to your computer via USB, then find the name of its serial port:

- Windows: open Device Manager, under "Ports (COM & LPT)", look for something like COM3.
- macOS: it usually appears as /dev/cu.wchusbserial... or /dev/cu.usbserial...
- Linux: it usually appears as /dev/ttyUSB0.

Erasing the old firmware

This removes everything on the chip, including the previous firmware, saved Wi-Fi credentials, and all settings. This is required when coming from ESPHome, and recommended for any firmware change.

```
python -m esptool --chip esp8266 --port <PORT> erase_flash
```

Flashing the new firmware

Download the latest firmware .bin file from GitHub (see chapter 10), then run:

```
esptool.py --chip esp8266 --port <PORT> --baud 115200 write_flash --flash_size=detect 0x0  
firmware.bin
```

Replace <PORT> with the serial port found in the previous step, and firmware.bin with the path to the downloaded firmware file. The flash mode and frequency do not need to be specified separately, they are already embedded in the firmware file.

Warning: Do not disconnect the Salt Sentry or interrupt the process while esptool is writing to the flash. Doing so can render the device unusable.

When flashing completes, unplug and reconnect the Salt Sentry. Because the flash was erased, it will start its Wi-Fi setup portal on first boot, follow chapter 3 to reconnect it to your network and restore your settings.

12. Programming the module with custom firmware

The Salt Sentry is based on the popular ESP8266 microprocessor (in an ESP-WROOM-02 package). If you want, you can program it with your own software, for example with the Arduino IDE or PlatformIO.

Connect the module to a computer via USB. If the module is not recognized, you may need to install USB-to-serial drivers for the CH340 chip.

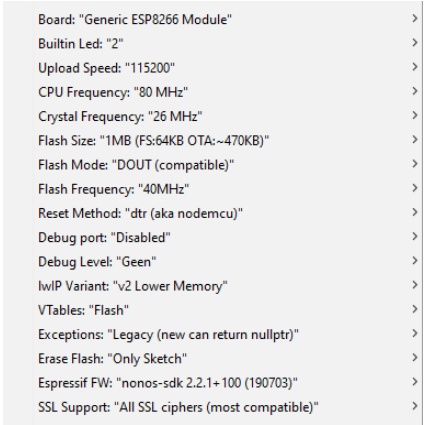
Arduino IDE

Use the following board settings in the Arduino IDE:

PlatformIO

Alternatively, the firmware project can be opened directly in PlatformIO. The relevant configuration:

```
[env:esp_wroom_02]
platform = espressif8266
board = esp_wroom_02
framework = arduino
monitor_speed = 115200
board_build.filesystem = littlefs
board_build.ldscript = eagle.flash.2m512.ld
build_flags = -I src
lib_deps =
    adafruit/Adafruit_VL53L0X@^1.2.5
    bblanchon/ArduinoJson@^5.13.0
```



Board: "Generic ESP8266 Module" >
Builtin Led: "2" >
Upload Speed: "115200" >
CPU Frequency: "80 MHz" >
Crystal Frequency: "26 MHz" >
Flash Size: "1MB (FS:64KB OTA:~470KB)" >
Flash Mode: "DOUT (compatible)" >
Flash Frequency: "40MHz" >
Reset Method: "dtr (aka nodemcu)" >
Debug port: "Disabled" >
Debug Level: "Geen" >
lwIP Variant: "v2 Lower Memory" >
VTables: "Flash" >
Exceptions: "Legacy (new can return nullptr)" >
Erase Flash: "Only Sketch" >
Espressif FW: "nonos-sdk 2.2.1+100 (190703)" >
SSL Support: "All SSL ciphers (most compatible)" >

Expansion pins

On the Salt Sentry there is room for a pin header with which you could expand the circuit with other sensors. The following pins are available:

- 3.3V
- GND
- GPIO02
- GPIO14
- GPIO04
- GPIO13
- GPIO16

The pin markings can be found on the bottom of the PCB.

The Salt Sentry uses the VL53L0X time-of-flight distance sensor, connected via I²C to GPIO2 (SDA) and GPIO14 (SCL) of the ESP-WROOM-02.

13. Using Salt Sentry with ESPHome

As an alternative to the stock Salt Sentry firmware, the module can also be flashed with ESPHome. This gives you a fully native Home Assistant integration through ESPHome's own API, without installing the Salt Sentry HACS integration, and full control over the configuration.

Warning: Flashing ESPHome replaces the stock Salt Sentry firmware entirely. Features that are specific to the stock firmware, such as the built-in Domoticz, OpenHAB, and MQTT configuration screens, and OTA updates through the Salt Sentry HACS integration, are not available once ESPHome is installed. This is only recommended for users who are already familiar with ESPHome. If you want to go back to the stock firmware, see chapter 11.

An example configuration is shown below. Replace the WiFi credentials with your own network (via ESPHome's secrets.yaml), and adjust the full_cm and empty_cm values to match your own water softener, the same way as described in chapter 3.

```
substitutions:
  name: "salt-sentry"

esphome:
  name: "${name}"
  project:
    name: lemcke-solutions.salt-sentry
    version: "v1.0"

esp8266:
  board: esp_wroom_02

# Enable logging
logger:

# Enable Home Assistant API
api:

ota:
  password: "<your own OTA password>"

wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password

# Enable fallback hotspot (captive portal) in case wifi connection fails
ap:
  ssid: "Salt-Sentry Fallback Hotspot"
  password: "<your own fallback AP password>"

captive_portal:

i2c:
  sda: 2
  scl: 14
  scan: true

globals:
  - id: full_cm
    type: float
    initial_value: '5'
```

```

- id: empty_cm
  type: float
  initial_value: '35'

sensor:
- platform: vl53l0x
  id: distance_m
  address: 0x29
  update_interval: 60s
  long_range: false
  internal: true
- platform: template
  unit_of_measurement: cm
  icon: mdi:arrow-expand-down
  name: distance
  id: distance
  lambda: |-
    return id(distance_m).state * 100;
- platform: template
  name: "percentage"
  unit_of_measurement: '%'
  icon: mdi:percent
  lambda: |-
    if (id(distance).state < id(full_cm)) {
      return 100;
    }

    if (id(distance).state > id(empty_cm)) {
      return 0;
    }

    return 100 - (id(distance).state - id(full_cm)) / ((id(empty_cm) - id(full_cm)) / 100);
  update_interval: 60s

```

Note: Choose your own OTA password and fallback hotspot password rather than reusing the example values above.

Once flashed, the device connects to Home Assistant automatically through the ESPHome integration, exposing a distance sensor and a percentage sensor.

Using inches instead of centimeters

If you prefer to work in inches, replace the globals: and sensor: sections above with the version below. Everything else in the configuration (wifi, api, ota, i2c, etc.) stays the same.

```

globals:
- id: full_in
  type: float
  initial_value: '2'
- id: empty_in
  type: float
  initial_value: '14'

sensor:
- platform: vl53l0x
  id: distance_m
  address: 0x29
  update_interval: 60s
  long_range: false

```

```

    internal: true
  - platform: template
    unit_of_measurement: in
    icon: mdi:arrow-expand-down
    name: distance
    id: distance
    lambda: |-
      return id(distance_m).state * 39.3701;
  - platform: template
    name: "percentage"
    unit_of_measurement: '%'
    icon: mdi:percent
    lambda: |-
      if (id(distance).state < id(full_in)) {
        return 100;
      }

      if (id(distance).state > id(empty_in)) {
        return 0;
      }

      return 100 - (id(distance).state - id(full_in)) / ((id(empty_in) - id(full_in)) / 100);
    update_interval: 60s

```

Note: Adjust the initial_value of full_in and empty_in to match your own water softener, measured in inches.

